

CAPSTONE PROJECT REPORT

Design, Simulation, and Hardening of a Secure Small Network

Robinson | Cybersecurity Portfolio Project

1. Project Overview & Network Design

This project implements a small office network built around defense in depth. Instead of one flat network where every device can reach every other device, the design uses a firewall and network segmentation to keep an untrusted device (an IoT camera) isolated from the production server and the admin workstation.

The lab was built entirely inside VMware Workstation, using pfSense Community Edition as the firewall/router, Kali Linux as the admin workstation and testing platform, and a VLAN to represent the untrusted IoT zone.

Network Design, In Plain Terms

The public internet connects into pfSense, which acts as the border firewall — it decides what traffic is allowed in and out. Inside the network, devices are split into zones instead of sharing one open network: the admin workstation and server sit in the trusted LAN, while the IoT camera sits on its own VLAN (IOT_ZONE). If the camera were compromised, the firewall rule and VLAN separation are what would stop an attacker from reaching the server or workstation from that camera.

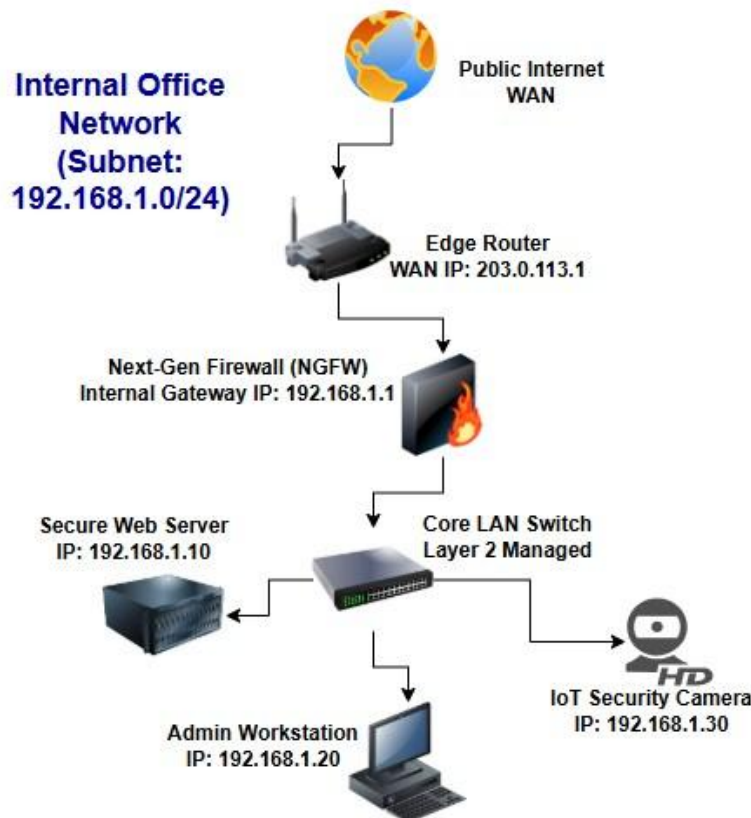


Figure 1 — Network topology: WAN → pfSense firewall → LAN switch → Server, Admin Workstation, IoT Camera

Asset & Addressing Table

Device / Role	Interface / Zone	Address
pfSense (Gateway/Firewall)	em0 (WAN) / em1 (LAN)	192.168.1.1
Kali Linux (Admin Workstation)	LAN	192.168.1.20
Secure Web Server (Production)	LAN	192.168.1.10
IoT Security Camera	IOT_ZONE (VLAN 30)	192.168.1.30 / 192.168.30.0-24

2. Lab Environment & Tools Used

The lab was built as an isolated virtual environment so testing could be done safely. The tools below were used to build, configure, and test it:

Tool	Type	Role in this project
VMware Workstation	Virtualization hypervisor	Hosted the entire lab environment — every VM (pfSense, Kali, and the network segments) running isolated on one laptop.
pfSense CE	Next-generation firewall/router	Acted as the gateway for the lab network: routing, firewall rule enforcement, and VLAN-based segmentation.
Kali Linux	Security-focused Linux distro	Served as the administrator workstation for configuring the network and as the platform used to run the simulated scan.
Nmap	Network port scanner	Scanned the pfSense gateway to identify open ports and simulate reconnaissance behaviour an attacker would use.
Wireshark	Packet analyzer	Captured live traffic on the Kali interface to inspect ARP resolution and TCP connection attempts at the packet level.

3. Security Controls Implemented

Five controls were implemented, covering initial hardening, traffic filtering, packet-level analysis, network segmentation, and access control.

Control 1 — Initial Hardening (Setup Wizard)

The pfSense setup wizard was used to move the firewall off its factory-default posture: hostname and time zone were set (for accurate log timestamps), DNS was pointed to 1.1.1.1 and 8.8.8.8, and the default admin password was replaced with a strong password.

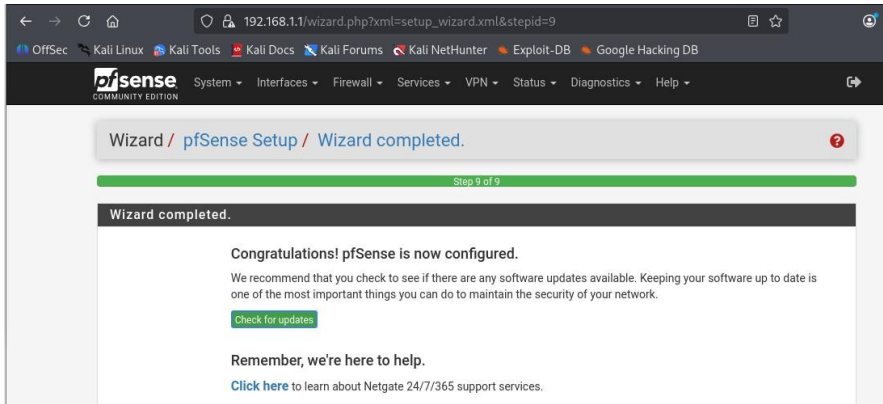


Figure 2 — pfSense setup wizard completed

Control 2 — Firewall Traffic Filtering

A block rule was created on the LAN interface targeting the IoT camera's address (192.168.1.30), intended to stop that device from reaching the rest of the LAN if it were ever compromised.

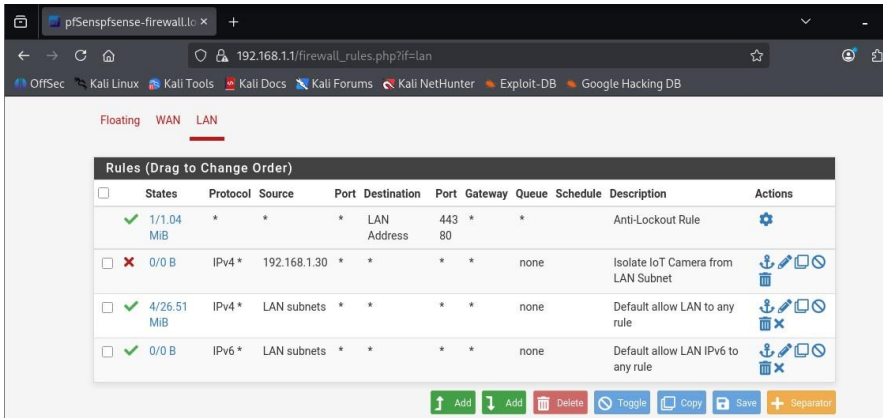


Figure 3 — LAN firewall rules, including the IoT isolation rule

Evidence note: The rule screenshot above shows the "Isolate IoT Camera from LAN Subnet" rule as disabled (red status, 0 B matched). Before final submission, enable the rule and generate traffic from the IoT zone so the state column shows it actively matching and blocking packets — this is what makes the control verifiable rather than just configured.

Control 3 — Packet-Level Traffic Analysis (Wireshark)

Wireshark was run on the Kali interface to capture live traffic and inspect it at the packet level — observing ARP resolution ("who has 192.168.1.1?") and repeated TCP SYN retransmissions toward an unresponsive host, which is a useful pattern to recognise when diagnosing connectivity or scan activity.

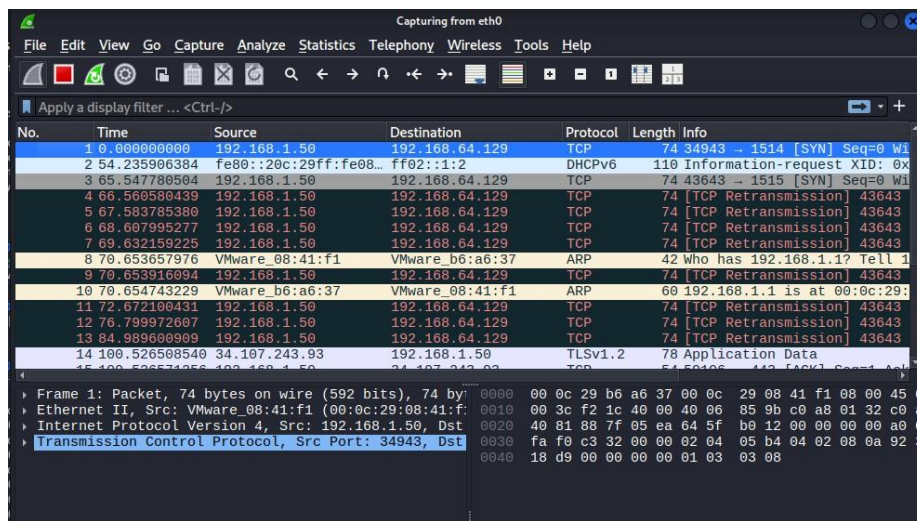


Figure 4 — Wireshark capture from eth0 showing ARP and TCP traffic

Control 4 — Network Segmentation (VLAN)

An 802.1Q VLAN (VLAN 30) was created on the LAN interface and assigned as a separate pfSense interface, IOT_ZONE, giving the IoT camera its own subnet and broadcast domain independent of the main LAN.

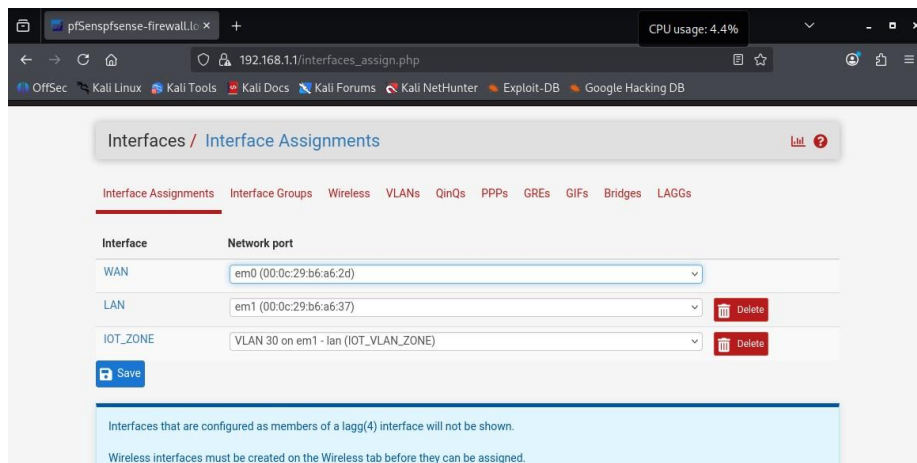


Figure 5 — Interface assignments showing IOT_ZONE as VLAN 30 on em1

Control 5 — User Access Control

A standard, non-administrative pfSense user account (auditor_robin) was created separately from the default admin account, so day-to-day log review does not require using the full-privilege admin login.

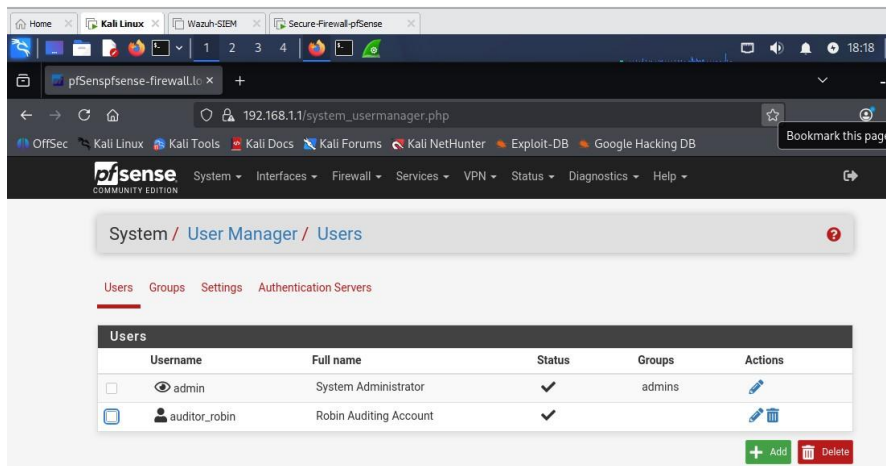


Figure 6 — pfSense User Manager showing the auditor_robin account

4. Simulated Threat & Response

Scenario: Network Reconnaissance

A common first step for an attacker on a network is reconnaissance — scanning to find live hosts and open services before deciding how to proceed. This was simulated from the Kali workstation using a fast Nmap scan against the pfSense gateway:

```
nmap -F 192.168.1.1
```

The scan identified three open ports on the gateway — 53 (DNS), 80 (HTTP), and 443 (HTTPS) — the services pfSense exposes for DNS resolution and web-based administration.

```
(robingens@kali)-[~]
$ nmap -F 192.168.1.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-08-14 17:08 +0100
Nmap scan report for 192.168.1.1
Host is up (0.0014s latency).
Not shown: 97 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https
MAC Address: 00:0C:29:B6:A6:37 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 2.35 seconds
```

Figure 7 — Nmap fast scan against the pfSense gateway (192.168.1.1)

```
(robingens@kali)-[~]
$ sudo ip addr add 192.168.1.20/24 dev eth0
Error: ipv4: Address already assigned.

(robingens@kali)-[~]
$ sudo ip addr flush dev eth0

(robingens@kali)-[~]
$ sudo ip addr add 192.168.1.20/24 dev eth0

(robingens@kali)-[~]
$ sudo ip route add default via 192.168.1.1

(robingens@kali)-[~]
$ ping -c 4 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data:
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=0.673 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=0.769 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=0.963 ms
64 bytes from 192.168.1.1: icmp_seq=4 ttl=64 time=0.911 ms

— 192.168.1.1 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3051ms
rtt min/avg/max/mdev = 0.673/0.829/0.963/0.114 ms
```

Figure 8 — Kali network configuration and connectivity test to the gateway

Evidence note: This scan was run from the authorized admin workstation directly against the gateway, so it demonstrates the reconnaissance technique but does not by itself prove the IoT-zone block. To fully support the "defensive response" claim, repeat the scan from a host placed in IOT_ZONE targeting the LAN, with the Control 2 rule enabled, and capture the resulting drop in the pfSense firewall log (Status > System Logs > Firewall).

Intended Defensive Response

With Control 2 (firewall filtering) and Control 4 (VLAN segmentation) both in place, the design intent is that an identical scan launched from inside IOT_ZONE toward the LAN would be silently dropped at the firewall, rather than reaching its targets. Confirming this behaviour with a log screenshot is the remaining step noted above.

5. Reflection

What I Learned

Security controls work together, not in isolation — a strong password means little if a compromised IoT device can reach every other host on the same flat network. Building pfSense from scratch made the flow of traffic between zones concrete rather than theoretical, and configuring the rules and VLAN myself made it clear exactly what each control is responsible for stopping.

What I Would Improve Next Time

I would connect the pfSense firewall logs to my existing Wazuh SIEM so that a blocked cross-zone attempt generates a real-time alert rather than something I have to check manually. I would also complete the IoT-to-LAN scan test above before calling the project finished, so every control in this report is backed by a screenshot that shows it actually working, not just configured.

6. Appendix — SOHO Security Checklist

A simple checklist for securing a typical home or small-office network, based on the controls applied in this project:

☐ **1. Change Default Router Credentials**

Replace the factory-default admin username and password immediately — these are publicly known and targeted by automated attacks.

☐ **2. Isolate IoT Devices**

Place smart TVs, cameras, and smart-home devices on a separate guest network or VLAN so they cannot reach your main computers.

☐ **3. Enable WPA2-AES or WPA3**

Never run an open Wi-Fi network — use the strongest encryption your router supports.

☐ **4. Disable UPnP, WPS, and Remote Management**

These convenience features can let external or malicious software open holes in your firewall automatically.

☐ **5. Keep Firmware Updated**

Enable automatic updates on your router, firewall, and smart devices so known exploits get patched.

☐ **6. Enforce Least Privilege**

Use a standard account for daily tasks; reserve the admin login strictly for configuration changes.